LIVING WORLD

プログラマー向け実装設計書

サーバー・クライアント・AI・DB実装担当向け

- 文書ID：ENG-001 / バージョン：0.9（レビュー用ベースライン）
- 対象：採択済みLIVING WORLDプロジェクト / 対象読者：サーバー・クライアント・AI・DB実装担当向け
- 表記：［合意済］＝これまでのゲーム構想で指定された内容、［設計案］＝実装に向けた推奨、［要決定］＝担当者による承認が必要
- 実装完了・性能達成・事業収益を保証する資料ではない。数値は明記した場合を除き確定値ではない。

文書の目的

アーキテクチャ、データモデル、API、トランザクション、NPC、経済、戦闘、ログ、テスト、CI/CDを実装時の判断材料として提示する。

構成

01 推奨アーキテクチャと境界

02 サービス間契約・API設計

03 データベース論理設計

04 アイテム一意性・通貨台帳

05 ゲームサーバー・移動・戦闘同期

06 NPC行動エンジン・LOD

07 NPC記憶・関係・会話

08 クエスト生成・排他受注

09 職業適格性・希少性

10 採集・クラフト・製造履歴

11 市場・地域価格・注文処理

12 銀行・配送・重量・輸送

13 死亡・遺品・ダンジョン資産

14 犯罪・司法・ギルド・都市成長

15 UI画面一覧・状態設計

16 ログ・監視・障害復旧

17 開発規約・CI/CD・マイグレーション

18 プログラマー向け実装優先順位

01 推奨アーキテクチャと境界

［設計案］単一巨大プロセスではなく、ゲームループと永続経済を分離する。最初はモジュラーモノリスで開始し、負荷と責務に応じてサービス分割する。

- Client：入力・描画・UI・補間。権威判定なし。
- Gateway/Auth：接続、認証、レート制限。
- Realtime World：位置・戦闘・可視NPC・近接物理。
- Simulation：村・NPC・価格・事件の抽象更新。
- Economy/Ledger：通貨、アイテム、所有権、契約の確定。
- AI Orchestrator：推論予算、入力制限、提案検証。
- Persistence/Outbox：DB、イベント、スナップショット、監査。

実装フロー・擬似コード

- Command→validate→transaction→outbox→event bus→projections→client notification。
- クライアントの再送にはrequest_idで同じ結果を返す。

開発者受入テスト

- AI停止中でも移動・採集・売買・銀行が利用できる。
- 一部サービス停止後にイベントを再送しても二重決済しない。

02 サービス間契約・API設計

［設計案］同期APIはユーザーの操作確定、非同期イベントは世界への波及に用いる。OpenAPI・AsyncAPIで契約を管理する。

- POST /v1/commands/vault/deposit：requestId, actorId, vaultId, itemInstanceIds, expectedVersion。
- POST /v1/commands/quest/accept：requestId, questId, actorId, expectedQuestVersion。
- POST /v1/commands/craft/start：recipeId, workstationId, inputItemIds, requestId。
- GET /v1/regions/{id}/market：現在価格・在庫・最終更新時刻・閲覧権限。
- イベントにはeventId, aggregateId, aggregateVersion, causationId, correlationId, occurredAt, payloadVersionを含める。

実装フロー・擬似コード

- 失敗コード：INSUFFICIENT_FUNDS / ITEM_NOT_OWNED / VERSION_CONFLICT / CAPACITY_EXCEEDED / EXPIRED / RATE_LIMITED。
- イベントスキーマ変更時は後方互換・移行計画・再生テストを実施。

開発者受入テスト

- 同じrequestIdを10回送信しても確定操作は1回。
- 旧イベントバージョンを読める。

03 データベース論理設計

PostgreSQL等のトランザクションDBを第一候補とする。データベース製品の最終選定は負荷・運用体制を踏まえて決定する。

- players(id, version, level, job_id, village_id, reputation, stats_json)。
- npcs(id, village_id, role_id, goals_json, schedule_json, assets_ref, state_version)。
- npc_relations(src_npc_id, dst_actor_id, trust, fear, affinity, debt, updated_at)。
- item_instances(id, type_id, quality, durability, origin_event_id, owner_type, owner_id, location_id, version)。
- ledger_entries(id, tx_id, account_id, asset_type, delta, occurred_at)。
- vaults(id, village_id, owner_id, capacity); vault_entries(vault_id, item_id, state)。
- quests(id, issuer_id, capacity, accepted_count, escrow_id, deadline, state, version)。
- shipments(id, carrier_id, origin, destination, cargo_ref, status, contract_id, version)。
- events(id, type, aggregate_id, version, causation_id, payload_json)。

実装フロー・擬似コード

- UNIQUE(item_instances.id); CHECK(ledger balances policy); UNIQUE(event_id); UNIQUE(request_id, actor_id)。
- 大きな会話履歴は要約と原本を分け、保存期間を定める。

開発者受入テスト

- 負残高・二重所有・孤児配送がないことを定期照合。

04 アイテム一意性・通貨台帳

最重要不変条件は「アイテム実体の所在が1か所」「通貨移転の貸借が一致」「人工物が未承認生成されない」である。

- ItemInstanceは生成元イベント、製造者、品質、現在所有者、所在、状態、versionを持つ。
- 取引では元所有者の保有確認→versionロック→新所有者へ移転→履歴→outboxを単一DBトランザクションにする。
- 通貨は原則として複式仕訳にし、手数料・税・報酬の移転先を明示する。
- 天然資源の再生と人工物の製造は別の「発行イベント種別」で監査する。
- アイテム破損・消耗・廃棄には明示的な消滅イベントを記録する。

実装フロー・擬似コード

- BEGIN; SELECT item FOR UPDATE; assert owner; UPDATE item SET owner=target,version=version+1; INSERT transfer; INSERT outbox; COMMIT。
- 再試行では同一requestIdの結果を再利用する。

開発者受入テスト

- 同時取得100要求で成功は1件。
- 死亡と銀行預入が競合してもアイテムは重複しない。
- 市場売買の合計通貨変化が発行・消滅ルールと一致する。

05 ゲームサーバー・移動・戦闘同期

一人称視点のアクション戦闘は低遅延の近接処理が必要。ネットワーク方式は未決定だが、権威判定はサーバーに保持する。

- 移動入力はtick番号・時刻・入力値を送り、サーバーで速度・衝突・重量制約を検証する。
- クライアントは予測と補間を使い、サーバー状態との差分を補正する。
- 攻撃は射程・視線・スタミナ・クールダウン・装備耐久・陣営ルールを確認する。
- 魔法は詠唱時間・魔力・触媒・対象条件をチェックする。
- ダメージ・死亡・犯罪判定は同一イベントから派生させる。

実装フロー・擬似コード

- Input→server simulation→hit validation→damage→death?→asset transaction→crime event。

開発者受入テスト

- 改ざんした移動速度・攻撃速度を拒否。
- 同じ攻撃IDで二重ダメージなし。

06 NPC行動エンジン・LOD

近接NPCと遠隔NPCで更新頻度とモデルを変える。遠隔更新を省略しても、訪問時に資産・予定・関係の矛盾が生じないことが条件。

- Level 0：目視範囲、個体単位の移動・会話・戦闘。
- Level 1：同一村、スケジュールと重要イベントのみ詳細処理。
- Level 2：遠隔村、職業集計・在庫・人口・治安の集約更新。
- NPCの意思決定は優先度（生命→契約→生活→個人目標）と効用評価で処理する。
- LLMは台詞や非定型交渉の候補を生成し、金銭・権限・状態を直接更新できない。

実装フロー・擬似コード

- if visible: high_fidelity_tick(); elif village_active: scheduled_tick(); else: aggregate_tick()。

開発者受入テスト

- LOD切替でNPCが二重出現・瞬間移動しない。
- LLMが使えなくても食料消費と市場は進む。

07 NPC記憶・関係・会話

NPCの記憶は無制限の全文保存ではなく、重要イベント・関係・要約・最近の会話を分ける。

- 関係値は信頼・好意・恐怖・借り・敵意など多軸で、相手によって非対称。
- 記憶はevent_id, actors, salience, confidence, visibility, expiry, summaryを持つ。
- NPCの知識は目撃・噂・手紙・旅人から更新し、情報源と確信度を保存する。
- 会話コンテキストは個人記憶、村の事実、直近発話、職業、関係値を優先。
- プレイヤー入力やNPC発話に含まれる命令文を、サーバー権限を与える命令として扱わない。

実装フロー・擬似コード

- context = retrieve(npc_id, player_id, location, salience_budget); candidate=LLM(context); validate(candidate)。

開発者受入テスト

- NPCが未確認の遠方価格を確定情報として断言しない。
- AI出力にアイテム生成命令が含まれても拒否。

08 クエスト生成・排他受注

クエスト生成は世界イベント→需要の発見→依頼主の支払能力→対象者の適格性→公開範囲→契約成立に分ける。

- QuestTemplateにtrigger, prerequisites, issuer, reward_source, cap, deadline, fail_rulesを定義。
- 個別NPCの依頼は信頼・面識・評判・職業などの条件を持つ。
- 1人限定依頼の受注ではquest row lockとaccepted_countを原子的に更新。
- 報酬は依頼主またはギルドの資産をescrowへ事前確保する。
- 依頼主死亡・期限切れ・途中放棄・失敗・競合の補償規則を用意する。

実装フロー・擬似コード

- generate→check issuer balance→escrow→offer→atomic accept→proof→settle or refund。

開発者受入テスト

- 100人が同時受注しても定員超過しない。
- 依頼主残高不足なら無償の報酬は発生しない。

09 職業適格性・希少性

無垢から基礎職への早期導線を確保し、希少職は行動履歴・適性・評判・地域・指導者・確率で候補を決める。

- job_eligibilityにactor_id, job_id, evidence, mentor_id, cooldown, statusを持たせる。
- 基礎職は固定チュートリアル導線を用意し、希少職は複数の入口を設定。
- 善行で聖職者、研究で魔法使い、犯罪で盗賊等の候補重みが変わる。
- 同じ手順を踏んでも全員が同じ希少クエストにならないが、過度の運任せは救済する。
- 適性の完全な可視化は避け、NPCの噂や技能成長から間接的に示す。

実装フロー・擬似コード

- eligible = hard_constraints && evidence_score>=threshold; offer = weighted_draw(context, cooldown, diversity)。

開発者受入テスト

- 基礎職の平均解放時間を測定。
- 希少職の出現率・地域差・不公平感を監視。

10 採集・クラフト・製造履歴

人工物は原材料と設備・労働・時間を消費して生産する。製造イベントとアイテム生成イベントを1トランザクションに結びつける。

- recipeには投入品、設備、技能、作業時間、成功率、品質式、廃材、燃料を持たせる。
- 開始時の素材ロック、完了時の確定、失敗時の消費・返却を区別。
- 資源再生は生態容量と採集圧を基に更新。
- 装備は製作者・製造地・品質・修理履歴を保持。
- 生産の非同期処理ではキャンセルと切断後の復帰を定義する。

実装フロー・擬似コード

- reserve inputs→start job→finish timer→roll quality→consume inputs→mint outputs with provenance。

開発者受入テスト

- 素材不足・同時製造で増殖しない。
- 製造途中の切断から復帰できる。

11 市場・地域価格・注文処理

価格はLLMに計算させず、在庫・注文・生産・輸送・購買力に基づく決定的ロジックで更新する。

- 村×品目ごとにstock, bid, ask, production, demand, price, timestampを管理。
- 価格更新は売買、供給不足、輸送到着、事件発生時を契機とする。
- NPCは情報伝播遅延を持ち、全世界価格を即時には知り得ない。
- 価格操作対策として急騰急落制限・注文のキャンセル履歴・異常検知を用意。
- 売買成立時はアイテム移転と通貨移転を同時に確定する。

実装フロー・擬似コード

- desired = base * scarcity_factor * demand_factor; bounded = clamp(desired, floor, ceiling); smooth(prev,bounded)。
- 上記は式の形の提案であり、係数・上限・下限は経済シミュレーションで決定。

開発者受入テスト

- 在庫0から売却できない。
- 買い占めで価格が反応し、供給増で変化する。

12 銀行・配送・重量・輸送

銀行保管と輸送中の所有権・所在を明確に分離する。輸送中は危険があるが、銀行内の預入品は死亡・盗難・襲撃から保護する。

- 預入：player inventory→vault inventory、引出は逆方向の原子的移転。
- 配送：受付→集荷→輸送中→到着→受領／事故・紛失・補償。
- 積載率は重量と体積の両方で判定し、移動速度とスタミナへ影響。
- 馬・荷馬・乗合馬車・貸切馬車・荷馬車・船を拡張可能なTransportTypeとして定義。
- 配送の補償条件と保険料は契約時に固定し、事故後に変更しない。

実装フロー・擬似コード

- deposit and death compete on item.version; only one transaction commits。
- shipment transfer requires cargo state=IN_TRANSIT; destination vault receives once。

開発者受入テスト

- 同時預入・死亡・引出で二重所有なし。
- 輸送中の荷物は別村で早期受領できない。

13 死亡・遺品・ダンジョン資産

死亡時の携行品は全てプレイヤーの所有から移り、ダンジョン資産プールや地面・モンスターに移転する。レベルは2低下、金銭の一部を失う。

- 死亡イベントのスナップショットで携行品を列挙し、個別アイテムIDを移転。
- ダンジョンプールは「人工物の実物在庫」と「価値の参考指標」を分ける。
- 宝箱の中身はプールに存在するアイテムのみを確保する。
- モンスター取得でアイテムをプールから移動し、装備効果を戦闘へ反映。
- 宝箱開封・モンスター討伐・遺品回収が競合した場合、versionロックで排他する。

実装フロー・擬似コード

- death tx: move carried items, apply level=max(minLevel,level-2), move cash by policy, emit loss event。

開発者受入テスト

- 死亡前後のアイテムID集合が保存される。
- 空プールから人工剣を生成しない。

14 犯罪・司法・ギルド・都市成長

施設破壊、窃盗、PvP、賠償、修繕依頼を同じイベント連鎖で扱う。

- CrimeCaseに行為者・被害者・場所・証拠・目撃・損害額・管轄・状態を持つ。
- 建築物は耐久・所有・修理必要素材・施工状況を保持。
- ギルド契約には発注者、資金拘束、受注上限、証拠、履行、清算を持つ。
- 都市の人口・治安・生産・需要・道路・建築を周期集計し、移住と建設を判断。
- 誤認・取消・冤罪・依頼主死亡などの補償手続きを設ける。

実装フロー・擬似コード

- WALL_DAMAGED→ASSESSMENT→PUBLIC_WORKS_REQUEST→ESCROW→GUILD_CONTRACT→MATERIAL_DELIVERY→REPAIR。

開発者受入テスト

- 壁の修理で資材が消費され、賃金が支払われる。
- 冤罪取消で手配状態が解消する。

15 UI画面一覧・状態設計

HUDの情報密度を制御し、重大なリスクを確定操作前に説明する。

- 画面：タイトル/設定/キャラ作成/HUD/地図/対話/ジョブ/スキル/インベントリ/クラフト。
- 画面：市場/売買確認/銀行/配送見積/ギルド/契約/クエスト/死亡/回収。
- 市場画面は現地価格・在庫・運賃・税・予想利益を区別。
- 死亡時は喪失した携行品、レベル変化、銀行保管の安全を表示。
- キーボード/マウスとゲームパッドの対応、字幕、文字サイズは要決定。

実装フロー・擬似コード

- 操作→確認ダイアログ→サーバー応答→成功表示 or 競合による再読込。

開発者受入テスト

- 失敗時に操作が完了したと誤認させない。
- 文字サイズ変更で主要画面が崩れない。

16 ログ・監視・障害復旧

運用上の最低要件は「誰が何を変更したか追える」「資産を復元できる」「AI停止時も世界が破綻しない」。

- 構造化ログ：trace_id, actor_id, command_id, aggregate_id, result, latency。
- 監視：tick遅延、AI待ち行列、DBロック、通貨不整合、アイテム孤児、配送滞留。
- スナップショット＋イベントログによる再生手順を整備。
- 障害時は該当機能を安全停止し、クライアントに処理保留を表示。
- 管理者操作は多要素認証・権限分離・監査ログを必須にする。

実装フロー・擬似コード

- restore snapshot→replay events→reconcile item ownership and ledger→open traffic。

開発者受入テスト

- 障害再現テストで二重移転がない。
- 監査ログから任意のアイテムの来歴を再構成。

17 開発規約・CI/CD・マイグレーション

仕様ID、コード、テスト、データスキーマ、運用手順をリンクする。

- リポジトリ構成案：client / server / simulation / ai-gateway / shared-schemas / ops / docs。
- API・イベントスキーマは契約テストで後方互換を確認。
- CI：静的解析→単体→契約→統合→経済保存則→負荷スモーク→パッケージ。
- DB移行はexpand→migrate→contractの段階で実施。
- Feature Flagで新機能を村・プレイヤー単位に限定公開できるようにする。

実装フロー・擬似コード

- PRには仕様ID、影響範囲、テスト証跡、DB移行有無を記載。

開発者受入テスト

- 旧クライアント接続時の互換動作を確認。
- リリース途中の失敗で安全にロールバック可能。

18 プログラマー向け実装優先順位

先に資産台帳とサーバー権威を作り、その上に市場・死亡・銀行・クエストを積み重ねる。

- Sprint 0：リポジトリ、CI、認証、権威サーバー、イベント・ログ。
- Sprint 1：移動・採集・ItemInstance・Ledger。
- Sprint 2：製造・市場・銀行・基礎職。
- Sprint 3：NPC生活・会話・クエスト・配送。
- Sprint 4：戦闘・死亡・ダンジョン再流通。
- Sprint 5：ギルド・法・都市変化・希少職。
- Sprint 6：負荷・不正・長期運用・復旧。

開発者受入テスト

- 各スプリントで縦切りE2Eが実行できる。
- 未決定の性能目標を仮定して合格判定しない。